

Placement Portal Application (PPA)

MAD-II Project Report

Student Details

- Name: Srivalli Jonnala
- Roll Number: 24f2007617
- Email: 24f2007617@ds.study.iitm.ac.in

1. Project Overview

The Placement Portal Application (PPA) is a multi-role web application that streamlines campus recruitment for an educational institute. It supports three roles — Admin (Institute Placement Cell), Company, and Student — each with a dedicated dashboard, and covers the full lifecycle of a placement drive: company onboarding, drive creation and approval, student applications, shortlisting, and final selection.

The problem statement asked for a system to replace spreadsheet- and email-based coordination between institutes, companies and students. My approach was to build a clean REST API in Flask backed by a single unified User model with a role field, then layer role-specific profile tables (StudentProfile, CompanyProfile) on top, and build a single-page Vue front end that talks to that API. Every write action that companies or students can trigger (registration, drive creation) is gated behind an admin approval step, matching the real-world workflow described in the brief.

2. Architecture & Tech Stack

Layer	Technology	Purpose
Backend API	Flask 3.x + Flask-SQLAlchemy	REST endpoints, ORM models, session-based auth
Frontend	Vue 3 (CDN, ES modules) + Bootstrap 5	SPA-style UI; Jinja2 used only as the single HTML entry point
Database	SQLite	Persistent relational data store, created programmatically
Cache	Redis + Flask-Caching	In-memory cache for read-heavy API responses
Task Queue	Celery 5 + Redis broker/backend	Scheduled and user-triggered async background jobs
Notifications	SMTP email + Google Chat webhook	Falls back to console logging when unconfigured, for local demos
Templating	Jinja2	Renders exactly one HTML shell (index.html); no other server-side templating

2.1 Project Structure

- `app/__init__.py` — Flask application factory; wires up SQLAlchemy, Flask-Caching, and all blueprints
- `app/config.py` — Central configuration: SQLite path, Redis URLs, Celery broker/backend, SMTP/webhook settings, seeded admin credentials
- `app/models/` — SQLAlchemy models: User, StudentProfile, CompanyProfile, Drive, Application
- `app/routes/` — Blueprints: auth, admin, company, student (all REST APIs) and pages (single catch-all SPA route)
- `app/tasks/` — Celery app + beat schedule, and the three background jobs: reminders, reports, export

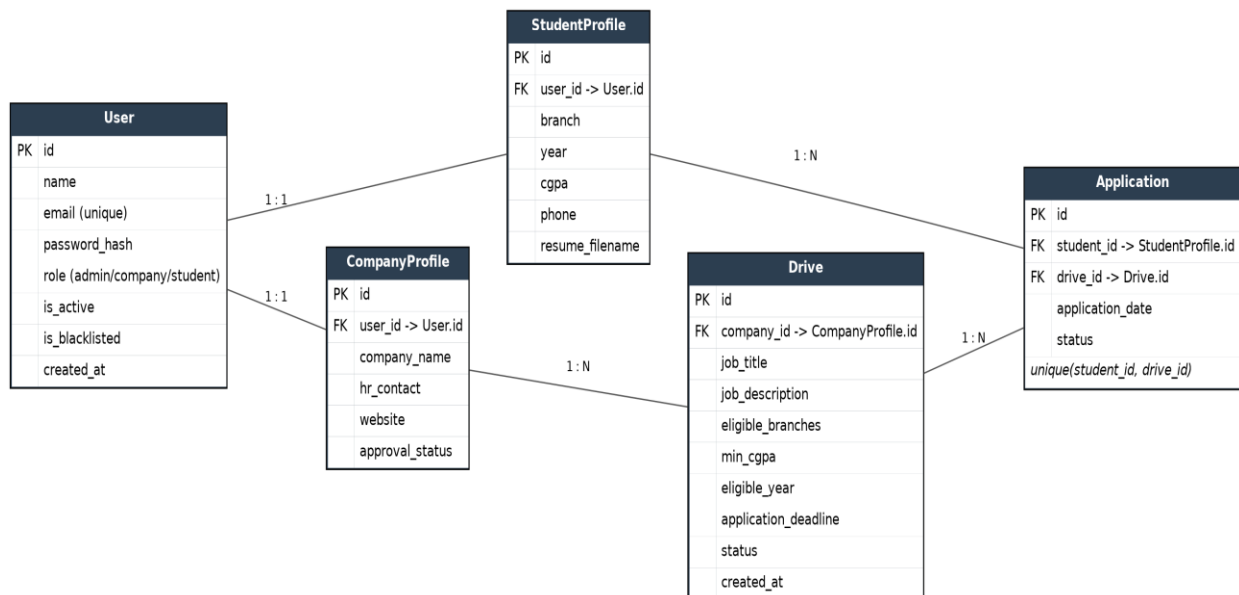
- app/utils/ — decorators.py (login_required / role_required) and notify.py (email + Google Chat helpers)
- app/static/js/ — main.js, app.js, router.js, api.js, components/ and pages/ (one Vue component per page)
- app/templates/index.html — the only Jinja2 template; a static HTML shell with an import map
- create_db.py — creates all tables via SQLAlchemy models and seeds the single admin user (no admin registration route exists)
- celery_worker.py / celery_beat.py — entry points for the Celery worker and beat scheduler

3. Database Design

The application uses a single SQLite database file (instance/ppa.db). All tables are defined via SQLAlchemy model classes and created programmatically by calling db.create_all() from create_db.py — no manual database tooling is used at any point.

Model	Key Fields	Relationships
User	id, name, email, password_hash, role, is_active, is_blacklisted	has one StudentProfile or CompanyProfile depending on role
StudentProfile	id, user_id, branch, year, cgpa, phone, resume_filename	belongs to User; has many Applications
CompanyProfile	id, user_id, company_name, hr_contact, website, approval_status	belongs to User; has many Drives
Drive	id, company_id, job_title, job_description, eligible_branches, min_cgpa, eligible_year, application_deadline, status	belongs to CompanyProfile; has many Applications
Application	id, student_id, drive_id, application_date, status	belongs to StudentProfile and Drive; unique constraint on (student_id, drive_id)

3.1 Entity-Relationship Diagram



4. Backend Jobs

All three background jobs are implemented as Celery tasks. The two scheduled jobs are registered in the Celery beat schedule (`app/tasks/celery_app.py`); the CSV export job is triggered on demand from the student dashboard.

4a. Daily Reminder (Scheduled)

Runs every day at 09:00 UTC. It finds all approved Drives whose application deadline falls within the next 3 days, then notifies every student who has not yet applied to each such drive, via a Google Chat webhook message and an email. Both notification channels fall back to console logging when not configured, so the job still completes end-to-end in a local demo.

4b. Monthly Activity Report (Scheduled)

Runs on the 1st of every month at 06:00 UTC. It aggregates the previous month's drives, applications, and selections into an HTML summary table and emails it to the institute admin.

4c. CSV Export (User-Triggered Async)

Triggered when a student clicks "Export Application History (CSV)" on their dashboard. The API immediately returns a `task_id` and 202 Accepted; the Celery worker builds a CSV of the student's applications (Student ID, Company Name, Drive Title, Status, Date) server-side and emails a completion alert. The frontend can poll `GET /api/student/export/status/<task_id>` to check job state.

5. Caching Strategy

Flask-Caching is configured with a Redis backend (`app/extensions.py`), so every blueprint can import and use the same cache instance without circular-import issues. Read-heavy, list-style endpoints are cached with a short TTL; any admin or company write action that could change the underlying data clears the cache so the next read is fresh.

Endpoint	TTL	Invalidated when
<code>GET /api/admin/stats</code>	30 s	Any admin approval/reject/blacklist action
<code>GET /api/student/drives</code>	30 s	TTL expiry, or a drive/company being approved

6. API Reference

6.1 Auth

Method	Endpoint	Description
POST	<code>/api/auth/register/student</code>	Register a student account
POST	<code>/api/auth/register/company</code>	Register a company account (pending admin approval)
POST	<code>/api/auth/login</code>	Session login
POST	<code>/api/auth/logout</code>	Clear session
GET	<code>/api/auth/me</code>	Current user + role-specific profile

6.2 Student

Method	Endpoint	Description
GET / PUT	<code>/api/student/profile</code>	View / update student profile

Method	Endpoint	Description
POST	/api/student/resume	Upload resume (PDF/DOC/DOCX)
GET	/api/student/drives	List approved drives (cached, searchable by title/branch)
POST	/api/student/drives/<id>/apply	Apply to a drive (eligibility + duplicate checked)
GET	/api/student/applications	Application history
POST	/api/student/export	Trigger async CSV export of application history
GET	/api/student/export/status/<task_id>	Poll Celery task status

6.3 Company

Method	Endpoint	Description
GET / PUT	/api/company/profile	View / update company profile
GET / POST	/api/company/drives	List / create placement drives (creation requires admin approval)
GET	/api/company/drives/<id>/applications	View applicants for a drive
PUT	/api/company/applications/<id>/status	Shortlist / select / reject an applicant

6.4 Admin

Method	Endpoint	Description
GET	/api/admin/stats	Dashboard counts (cached)
GET	/api/admin/companies	List / search companies
POST	/api/admin/companies/<id>/approve	Approve a company
POST	/api/admin/companies/<id>/reject	Reject a company
POST	/api/admin/companies/<id>/blacklist	Toggle company blacklist
GET	/api/admin/students	List / search students
POST	/api/admin/students/<id>/toggle-active	Toggle student active/deactivated status
GET	/api/admin/drives	List all drives, optionally filtered by status
POST	/api/admin/drives/<id>/approve	Approve a drive
POST	/api/admin/drives/<id>/reject	Reject a drive
GET	/api/admin/applications	All applications across the platform

AI/LLM Declaration

I used Claude.ai to assist with improving documentation wording, and clarifying certain backend concepts. I used ChatGPT-5 to generate basic frontend buttons and div code, which I reviewed and integrated into the project. All core implementation, database design decisions, debugging, and system integration were completed independently by me.

Video Presentation

Drive link: <https://drive.google.com/file/d/1TvGpb96y37A3LWIT-rB0q2ZR2Un7YDNG/view?usp=sharing>